INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARÍA DE INGENIERÍA Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

Práctica º
"Blowfish"

Fecha de entrega: 04-05-2026

NOMBRES:

Pérez Vázquez María Fernanda

Perucho Solis Kevin

García Pérez Cristofer Alexander

ASIGNATURA: Virología y criptografía

SECUENCIA: 6NM67

PROFESOR: Víctor Garduño Mendieta

Introducción

En la presente práctica se implementa el algoritmo de cifrado simétrico Blowfish mediante el lenguaje de programación Python. Blowfish es un cifrador de bloques de 64 bits diseñado en 1993 por Bruce Schneier como una alternativa rápida y libre a los algoritmos existentes como DES y IDEA.

El programa desarrollado permite al usuario cifrar y descifrar texto de forma interactiva desde la línea de comandos, utilizando el modo CBC (Cipher Block Chaining) y una derivación segura de clave mediante PBKDF2-SHA256. El objetivo principal es comprender el funcionamiento práctico del cifrado simétrico y sus mecanismos de seguridad.

Índice

Introducción 2
Consideraciones teóricas 4
Material 4
Desarrollo 5
  Análisis 5
  Código original 13
  Código modificado 14
Observaciones 21
Conclusiones 22
Bibliografía 23
Anexos 24

Consideraciones teóricas

Blowfish es un algoritmo de cifrado por bloques de clave simétrica con las siguientes características:

•Tamaño de bloque: 64 bits (8 bytes).

•Longitud de clave variable: entre 32 y 448 bits.

•Estructura: red Feistel de 16 rondas.

•Operaciones: XOR, suma modular y sustitución mediante S-boxes.

•Uso de subclave y cuatro S-boxes de 8x32 bits precalculadas a partir de la clave.

En esta práctica se emplean los siguientes mecanismos adicionales de seguridad:

•Modo CBC (Cipher Block Chaining): cada bloque se combina con el bloque cifrado anterior antes de cifrarse, evitando patrones repetitivos.

•Vector de inicialización (IV): valor aleatorio de 8 bytes que garantiza que el mismo texto con la misma clave produzca cifrados distintos.

•PBKDF2-SHA256: función de derivación de clave con 100,000 iteraciones y salt aleatorio de 16 bytes para proteger contra ataques de diccionario.

•Padding PKCS7: rellena el texto para que sea múltiplo del tamaño de bloque.

•Codificación Base64: permite representar los datos cifrados (binarios) como texto ASCII.

Material

Para la realización de esta práctica se utilizaron los siguientes recursos:

Computadora con sistema operativo Windows 10/11.

•Python 3.13 instalado y configurado en el PATH del sistema.

•Visual Studio Code como entorno de desarrollo integrado (IDE).

•Biblioteca pycryptodome versión 3.23.0 (instalada mediante pip).

•Terminal integrada de Visual Studio Code (PowerShell).

Instalación de la dependencia requerida:

py -m pip install pycryptodome

Desarrollo

Análisis

El programa está desarrollado en Python y se divide en tres módulos funcionales bien definidos:

- Núcleo Blowfish: contiene la lógica criptográfica pura (derivación de clave, cifrado y descifrado).
- Capa de texto: convierte entre texto plano y formato Base64 para facilitar el manejo del resultado.
- Interfaz de usuario: menú interactivo en terminal que guía al usuario paso a paso.

2. Derivación de clave — derivar_clave()

El usuario ingresa una contraseña legible como "miclave123", pero Blowfish requiere una clave binaria de longitud fija. Para esto se usa PBKDF2-SHA256 (Password-Based Key Derivation Function 2):

```python
def derivar_clave(password: str, salt: bytes) -> bytes:
    """Deriva una clave de 32 bytes desde una contraseña usando PBKDF2-SHA256."""
    return hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100_000, dklen=32)
```

La contraseña se convierte a bytes con .encode().

Se combina con un salt aleatorio de 16 bytes para que dos usuarios con la misma contraseña obtengan claves distintas.

Se aplican 100,000 iteraciones del hash SHA-256, lo que hace computacionalmente costoso adivinar la contraseña por fuerza bruta.

El resultado es una clave de exactamente 32 bytes (256 bits).

3. Proceso de cifrado — cifrar()

Esta función realiza los siguientes pasos en orden:

```python
def cifrar(datos: bytes, password: str) -> bytes:
    """
    Cifra datos con Blowfish (modo CBC + padding PKCS7).
    Formato del paquete:
        [4 bytes: longitud del salt] + [salt (16 bytes)] + [IV (8 bytes)] + [datos cifrados]
    """
    salt   = get_random_bytes(16)
    clave  = derivar_clave(password, salt)
    iv     = get_random_bytes(8)
    cipher = Blowfish.new(clave, Blowfish.MODE_CBC, iv)
    cifrado = cipher.encrypt(pad(datos, Blowfish.block_size))
    return struct.pack(">I", 16) + salt + iv + cifrado
```

Paso 1 — Generar valores aleatorios:

salt = get_random_bytes(16) # 16 bytes aleatorios

iv = get_random_bytes(8) # 8 bytes aleatorios (tamaño de bloque Blowfish)

El salt protege la contraseña. El IV (Vector de Inicialización) garantiza que el mismo texto cifrado dos veces produzca resultados distintos.

Paso 2 — Derivar la clave:

clave = derivar_clave(password, salt)

Paso 3 — Crear el objeto cifrador:

cipher = Blowfish.new(clave, Blowfish.MODE_CBC, iv)

Se instancia Blowfish en **modo CBC** con la clave e IV generados.

Paso 4 — Aplicar padding y cifrar:

cifrado = cipher.encrypt(pad(datos, Blowfish.block_size))

Blowfish trabaja con bloques de exactamente 8 bytes. Si el texto no es múltiplo de 8, el **padding PKCS7** rellena el último bloque con bytes extra. Por ejemplo, si faltan 3 bytes, agrega \x03\x03\x03.

Paso 5 — Empaquetar el resultado:

return struct.pack(">I", 16) + salt + iv + cifrado

4. Modo CBC — ¿Por qué es importante?

En el modo CBC, cada bloque de texto plano se combina (XOR) con el bloque cifrado anterior antes de cifrarse:

Bloque 1: Texto_1 XOR IV $\rightarrow \text{Cifrado}_1$

Bloque 2: Texto_2 XOR Cifrado_1 $\rightarrow \text{Cifrado}_2$

Bloque 3: Texto_3 XOR Cifrado_2 $\rightarrow \text{Cifrado}_3$

Esto tiene dos ventajas clave:

- **Elimina patrones**: si el texto tiene palabras repetidas, los bloques cifrados serán completamente distintos entre sí.
- **Efecto avalancha**: un cambio en un bloque afecta todos los bloques siguientes.

Sin CBC (modo ECB), bloques iguales de texto producen bloques iguales de cifrado, lo que filtra información sobre el contenido.

5. Funcionamiento interno de Blowfish

Blowfish es un **cifrador de bloques con estructura de red Feistel de 16 rondas**. Su funcionamiento es el siguiente:

Preparación (key schedule): Antes de cifrar, Blowfish expande la clave en dos estructuras:

- **Array P**: 18 subclaves de 32 bits cada una.
- **S-boxes**: 4 tablas de sustitución de 256 entradas de 32 bits cada una.

Estas estructuras se inicializan con los dígitos de π (pi) y luego se modifican con la clave del usuario, lo que hace que Blowfish sea especialmente lento al cambiar de clave (ventaja de seguridad).

Cifrado de un bloque de 64 bits:

El bloque se divide en dos mitades de 32 bits: **L** (izquierda) y **R** (derecha). En cada ronda:

Para i = 1 hasta 16:

L = L XOR P[i]

R = F(L) XOR R

Intercambiar L y R

Intercambio final:

R = R XOR P[17]

L = L XOR P[18]

La función F es el corazón de Blowfish:

F(x) = ((S[0][a] + S[1][b]) XOR S[2][c]) + S[3][d]

Donde a, b, c, d son los 4 bytes del valor de 32 bits x. Combina sumas y XORs sobre las S-boxes para lograr confusión y difusión.

6. Proceso de descifrado — descifrar()

El descifrado extrae cada componente del paquete usando offsets:

```python
def descifrar(paquete: bytes, password: str) -> bytes:
    """Descifra un paquete generado por la función cifrar()."""
    offset   = 0
    salt_len = struct.unpack(">I", paquete[offset:offset+4])[0]; offset += 4
    salt     = paquete[offset:offset+salt_len];                  offset += salt_len
    iv       = paquete[offset:offset+8];                         offset += 8
    datos_cifrados = paquete[offset:]

    clave  = derivar_clave(password, salt)
    cipher = Blowfish.new(clave, Blowfish.MODE_CBC, iv)
    return unpad(cipher.decrypt(datos_cifrados), Blowfish.block_size)
```

salt_len = struct.unpack(">I", paquete[0:4])[0] # Lee longitud del salt

salt = paquete[4:20] # Extrae salt

iv = paquete[20:28] # Extrae IV

datos = paquete[28:] # Datos cifrados

Luego recalcula la clave con la contraseña ingresada y descifra:

cipher = Blowfish.new(clave, Blowfish.MODE_CBC, iv)

return unpad(cipher.decrypt(datos), Blowfish.block_size)

Si la contraseña es incorrecta, la función unpad() lanza una excepción porque el padding resultante no tiene el formato PKCS7 esperado. Esta excepción se captura en la interfaz mostrando el mensaje de error al usuario.

7. Codificación Base64

Como el resultado del cifrado son bytes binarios, no pueden mostrarse ni copiarse fácilmente en la terminal. Por eso se convierten a **Base64**, un esquema que representa datos binarios usando solo 64 caracteres ASCII imprimibles (A-Z, a-z, 0-9, +, /).

Al cifrar:

base64.b64encode(paquete).decode("ascii")

Al descifrar:

base64.b64decode(texto_b64.encode("ascii"))

Esto permite que el usuario pueda copiar, pegar y compartir el texto cifrado como si fuera texto normal.

Cifrar

```python
def cifrar_texto(texto: str, password: str) -> str:
    """Cifra un texto y devuelve el resultado en Base64."""
    paquete = cifrar(texto.encode("utf-8"), password)
    return base64.b64encode(paquete).decode("ascii")
```

texto.encode("utf-8") → convierte el texto a bytes (las computadoras trabajan con bytes, no con letras)

cifrar(...) → llama al núcleo Blowfish y regresa el paquete binario cifrado

base64.b64encode(paquete) → convierte esos bytes binarios a texto legible (Base64)

.decode("ascii") → convierte el resultado a string de Python para poder imprimirlo

```python
def descifrar_texto(texto_b64: str, password: str) -> str:
    """Descifra un texto cifrado en Base64 y devuelve el texto original."""
    paquete = base64.b64decode(texto_b64.encode("ascii"))
    return descifrar(paquete, password).decode("utf-8")
```

Hace exactamente lo contrario: Base64 → bytes → descifra → texto legible.

```python
def banner():
    print("""
╔═══════════════════════════════════════════════╗
║          BLOWFISH CIFRADOR DE TEXTO v1.0      ║
║  Algoritmo: Blowfish CBC + PBKDF2-SHA256      ║
╚═══════════════════════════════════════════════╝
""")
```

Solo imprime el encabezado decorativo cuando arranca el programa. No tiene lógica, es puramente visual.

main() — el menú principal

El loop infinito

```python
def main():
    banner()
    while True:
        print("  [1] Cifrar texto")
        print("  [2] Descifrar texto")
        print("  [0] Salir")
        opcion = input("\n  Elige opción: ").strip()
```

El programa corre **indefinidamente** hasta que el usuario elige 0. .strip() elimina espacios accidentales que el usuario pueda escribir.

Opción 1 — Cifrar

```python
    if opcion == "1":
        print("\n── CIFRAR ──────────────────────────────────────")
        texto    = input("  Texto a cifrar: ")
        password = getpass("  Contraseña: ")
        resultado = cifrar_texto(texto, password)
        print("\n  ✓ Texto cifrado (Base64):\n")
        for i in range(0, len(resultado), 64):
            print("  " + resultado[i:i+64])
        print()
```

getpass() → pide la contraseña sin mostrarla en pantalla (como cuando escribes en un banco)

El for con range(0, len(resultado), 64) → parte el Base64 en líneas de 64 caracteres para que se vea ordenado, por ejemplo:

ABCDefgh1234... XYZabcde5678...

Opción 2 — Descifrar

Como el texto cifrado se mostró en varias líneas de 64 caracteres, aquí el usuario las pega todas y el programa las **junta en una sola cadena** antes de descifrar. El loop termina cuando el usuario presiona Enter en una línea vacía.

```python
elif opcion == "2":
    print("\n── DESCIFRAR ──────────────────────────────────────")
    print("  Pega el texto cifrado y presiona Enter dos veces:")
    lineas = []
    while True:
        linea = input()
        if linea == "":
            break
        lineas.append(linea.strip())
    cifrado_b64 = "".join(lineas)
    password = getpass("  Contraseña: ")
    try:
        resultado = descifrar_texto(cifrado_b64, password)
        print(f"\n  ✓ Texto descifrado:\n  {resultado}\n")
    except Exception:
        print("\n  ✗ Error: contraseña incorrecta o datos corruptos.\n")

elif opcion == "0":
    print("\n  Hasta luego. Mantén tus datos seguros 🔒\n")
    break
else:
    print("  Opción no válida.\n")
```

El try/except atrapa cualquier error (contraseña incorrecta, datos dañados) y muestra un mensaje amigable en lugar de que el programa se rompa.

```python
try:
    resultado = descifrar_texto(cifrado_b64, password)
    print(f"\n  ✓ Texto descifrado:\n  {resultado}\n")
except Exception:
    print("\n  ✗ Error: contraseña incorrecta o datos corruptos.\n")
```

Opción 0 — Salir

python

Rompe el while True y termina el programa.

```python
if __name__ == "__main__":
    main()
```

Esto significa: **"solo ejecuta main() si estás corriendo este archivo directamente"**. Si alguien importara este archivo como librería desde otro script, main() no se ejecutaría automáticamente.

Código original

```python
"""
                                    BLOWFISH original
==================================Cifra texto=====================================
en el terminal se debe instalar en la consola "pip install pycryptodome" para el uso.
==================================================================================
"""

import sys
import base64
import hashlib
import struct
from getpass import getpass

try:
    from Crypto.Cipher import Blowfish
    from Crypto.Random import get_random_bytes
    from Crypto.Util.Padding import pad, unpad
except ImportError:
    print("\n[ERROR] Instala la dependencia con:\n    pip install pycryptodome\n")
    sys.exit(1)


# ──────────────────────────────────────────────
#  NÚCLEO BLOWFISH
# ──────────────────────────────────────────────

def derivar_clave(password: str, salt: bytes) -> bytes:
    """Deriva una clave de 32 bytes desde una contraseña usando SHA-256 + salt."""
    return hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100_000, dklen=32)


def cifrar(datos: bytes, password: str) -> bytes:
    """
    Cifra datos arbitrarios con Blowfish (modo CBC + padding PKCS7).
    Formato del paquete cifrado:
        [4 bytes: longitud del salt] + [salt (16 bytes)] + [IV (8 bytes)] + [datos cifrados]
    """
    salt = get_random_bytes(16)
    clave = derivar_clave(password, salt)
    iv = get_random_bytes(8)                        # Blowfish usa bloques de 8 bytes
    cipher = Blowfish.new(clave, Blowfish.MODE_CBC, iv)
    datos_padded = pad(datos, Blowfish.block_size)
    cifrado = cipher.encrypt(datos_padded)

    paquete = struct.pack(">I", 16) + salt + iv + cifrado
    return paquete


def descifrar(paquete: bytes, password: str) -> bytes:
    """Invierte el proceso de cifrado y devuelve los datos originales."""
    offset = 0
    salt_len = struct.unpack(">I", paquete[offset:offset+4])[0]
    offset += 4

    salt = paquete[offset:offset+salt_len]
    offset += salt_len

    iv = paquete[offset:offset+8]
    offset += 8

    datos_cifrados = paquete[offset:]

    clave = derivar_clave(password, salt)
    cipher = Blowfish.new(clave, Blowfish.MODE_CBC, iv)
    datos_padded = cipher.decrypt(datos_cifrados)
    return unpad(datos_padded, Blowfish.block_size)

# ──────────────────────────────────────────────
#  CIFRADO / DESCIFRADO DE TEXTO
# ──────────────────────────────────────────────

def cifrar_texto(texto: str, password: str) -> str:
    """Cifra un texto y devuelve el resultado en Base64."""
    paquete = cifrar(texto.encode("utf-8"), password)
    return base64.b64encode(paquete).decode("ascii")


def descifrar_texto(texto_b64: str, password: str) -> str:
    """Descifra un texto cifrado en Base64 y devuelve el texto original."""
    paquete = base64.b64decode(texto_b64.encode("ascii"))
    return descifrar(paquete, password).decode("utf-8")


# ──────────────────────────────────────────────
#  INTERFAZ
# ──────────────────────────────────────────────

def banner():
    print("\n Algoritmo: Blowfish CBC + PBKDF2-SHA256 \n")


def main():
    banner()
    while True:
        print("  [1] Cifrar texto")
        print("  [2] Descifrar texto")
        print("  [0] Salir")
        opcion = input("\n  Elige opción: ").strip()

        if opcion == "1":
            print("\n── CIFRAR ──────────────────────────────────────────")
            texto    = input("  Texto a cifrar: ")
            password = getpass("  Contraseña: ")
            resultado = cifrar_texto(texto, password)
            print("\n  Texto cifrado (Base64):\n")
            for i in range(0, len(resultado), 64):
                print("  " + resultado[i:i+64])
            print()

        elif opcion == "2":
            print("\n── DESCIFRAR ───────────────────────────────────────")
            print("  Pega el texto cifrado y presiona Enter dos veces:")
            lineas = []
            while True:
                linea = input()
                if linea == "":
                    break
                lineas.append(linea.strip())
            cifrado_b64 = "".join(lineas)
            password = getpass("  Contraseña: ")
            try:
                resultado = descifrar_texto(cifrado_b64, password)
                print(f"\n  Texto descifrado:\n  {resultado}\n")
            except Exception:
                print("\n  Error: contraseña incorrecta o datos corruptos.\n")

        elif opcion == "0":
            print("\n  Hasta luego. Mantén tus datos seguros \n")
            break
        else:
            print("  Opción no válida.\n")


if __name__ == "__main__":
    main()
```

Código modificado

Teniendo en cuenta el código original lo que vamos a integrar en ese mismo código es integrar la parte de encriptar y desencriptar imágenes y el archivo.

Para esto se debe explicar el siguiente código:

```python
import os
import sys
import json
import base64
import hashlib
import struct
from getpass import getpass
from datetime import datetime

try:
    from Crypto.Cipher import Blowfish
    from Crypto.Random import get_random_bytes
    from Crypto.Util.Padding import pad, unpad
except ImportError:
    print("\n[ERROR] Instala la dependencia con:\n    pip install pycryptodome\n")
    sys.exit(1)
```

Aquí se menciona las librerías que se van a usar en el programa

```python
# ──────────────────────────────────────────────
#  NÚCLEO BLOWFISH
# ──────────────────────────────────────────────

def derivar_clave(password: str, salt: bytes) -> bytes:
    """Deriva una clave de 32 bytes desde una contraseña usando SHA-256 + salt."""
    return hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 100_000, dklen=32)


def cifrar(datos: bytes, password: str) -> bytes:
    """
    Cifra datos arbitrarios con Blowfish (modo CBC + padding PKCS7).
    Formato del paquete cifrado:
        [4 bytes: longitud del salt] + [salt (16 bytes)] + [IV (8 bytes)] + [datos cifrados]
    """
    salt = get_random_bytes(16)
    clave = derivar_clave(password, salt)
    iv = get_random_bytes(8)                        # Blowfish usa bloques de 8 bytes
    cipher = Blowfish.new(clave, Blowfish.MODE_CBC, iv)
    datos_padded = pad(datos, Blowfish.block_size)
    cifrado = cipher.encrypt(datos_padded)

    paquete = struct.pack(">I", 16) + salt + iv + cifrado
    return paquete
```

```python
def descifrar(paquete: bytes, password: str) -> bytes:
    """Invierte el proceso de cifrado y devuelve los datos originales."""
    offset = 0
    salt_len = struct.unpack(">I", paquete[offset:offset+4])[0]
    offset += 4

    salt = paquete[offset:offset+salt_len]
    offset += salt_len

    iv = paquete[offset:offset+8]
    offset += 8

    datos_cifrados = paquete[offset:]

    clave = derivar_clave(password, salt)
    cipher = Blowfish.new(clave, Blowfish.MODE_CBC, iv)
    datos_padded = cipher.decrypt(datos_cifrados)
    return unpad(datos_padded, Blowfish.block_size)
```

Convierte tu contraseña normal en una clave criptográfica segura. Y los encripta y desencripta

Estos bloques se usan para que el sistema de cifrado Blowfish trabaje con **3 tipos de datos**:

```python
# ────────────────────────────────────────────────────────────
#  1. CIFRADO DE TEXTO
# ────────────────────────────────────────────────────────────

def cifrar_texto(texto: str, password: str) -> str:
    """Cifra texto y devuelve el resultado en Base64 (fácil de copiar/pegar)."""
    datos = texto.encode("utf-8")
    paquete = cifrar(datos, password)
    return base64.b64encode(paquete).decode("ascii")


def descifrar_texto(texto_b64: str, password: str) -> str:
    """Descifra texto previamente cifrado con cifrar_texto."""
    paquete = base64.b64decode(texto_b64.encode("ascii"))
    datos = descifrar(paquete, password)
    return datos.decode("utf-8")
```

Convierte texto normal en texto cifrado y viceversa

```python
# ────────────────────────────────────────────────────────────
#  2. CIFRADO DE IMÁGENES
# ────────────────────────────────────────────────────────────

def cifrar_imagen(ruta_entrada: str, ruta_salida: str, password: str) -> None:
    """
    Lee una imagen, la cifra con Blowfish y guarda el archivo cifrado.
    El archivo cifrado lleva extensión .bf para indicar que está cifrado.
    """
    if not os.path.isfile(ruta_entrada):
        raise FileNotFoundError(f"No se encontró el archivo: {ruta_entrada}")

    with open(ruta_entrada, "rb") as f:
        datos_imagen = f.read()

    # Guardamos la extensión original dentro del paquete para recuperarla al descifrar
    ext = os.path.splitext(ruta_entrada)[1].encode("utf-8")
    ext_padded = ext.ljust(10)[:10]         # 10 bytes fijos para la extensión
    datos_con_meta = ext_padded + datos_imagen

    paquete = cifrar(datos_con_meta, password)

    with open(ruta_salida, "wb") as f:
        f.write(paquete)

    print(f"  Imagen cifrada guardada en: {ruta_salida}")
```

```python
def descifrar_imagen(ruta_cifrada: str, ruta_salida: str, password: str) -> None:
    """Descifra una imagen previamente cifrada con cifrar_imagen."""
    with open(ruta_cifrada, "rb") as f:
        paquete = f.read()

    datos_con_meta = descifrar(paquete, password)

    ext = datos_con_meta[:10].rstrip().decode("utf-8")
    datos_imagen = datos_con_meta[10:]

    # Si no se especificó extensión en la salida, la añadimos
    if not os.path.splitext(ruta_salida)[1]:
        ruta_salida = ruta_salida + ext

    with open(ruta_salida, "wb") as f:
        f.write(datos_imagen)

    print(f" Imagen descifrada guardada en: {ruta_salida}")
```

Toma una imagen (.jpg, .png) y la convierte en archivo cifrado y viceversa

```python
# ────────────────────────────────────────────────────────────
# 3. CIFRADO DE DOCUMENTOS
# ────────────────────────────────────────────────────────────

def cifrar_documento(ruta_entrada: str, ruta_salida: str, password: str) -> None:
    """
    Cifra cualquier documento (PDF, Word, Excel, PowerPoint, etc.) con Blowfish.

    El paquete cifrado contiene:
        [10 bytes: extensión original] + [bytes del archivo]

    El archivo cifrado lleva extensión .bfdoc para indicar que está cifrado.
    """
    if not os.path.isfile(ruta_entrada):
        raise FileNotFoundError(f"No se encontró el archivo: {ruta_entrada}")

    ext = os.path.splitext(ruta_entrada)[1].lower()
    tipo = EXTENSIONES_DOCUMENTO.get(ext, "Archivo genérico")

    with open(ruta_entrada, "rb") as f:
        datos_doc = f.read()

    tam_mb = len(datos_doc) / (1024 * 1024)
    print(f"  Tipo detectado : {tipo} ({ext})")
    print(f"  Tamaño         : {tam_mb:.2f} MB ({len(datos_doc):,} bytes)")
    print(f"  Cifrando...    ", end="", flush=True)

    # Guardar extensión (10 bytes fijos) + contenido del archivo
    ext_bytes = ext.encode("utf-8").ljust(10)[:10]
    datos_con_meta = ext_bytes + datos_doc

    paquete = cifrar(datos_con_meta, password)

    with open(ruta_salida, "wb") as f:
        f.write(paquete)

    tam_cifrado = os.path.getsize(ruta_salida) / (1024 * 1024)
    print("listo.")
    print(f"  Documento cifrado guardado en : {ruta_salida}")
    print(f"  Tamaño cifrado                : {tam_cifrado:.2f} MB")
```

```python
def descifrar_documento(ruta_cifrada: str, ruta_salida: str, password: str) -> None:
    """
    Descifra un documento previamente cifrado con cifrar_documento.
    Restaura la extensión original si no se especificó en ruta_salida.
    """
    if not os.path.isfile(ruta_cifrada):
        raise FileNotFoundError(f"No se encontró el archivo: {ruta_cifrada}")

    print(f"  Descifrando... ", end="", flush=True)

    with open(ruta_cifrada, "rb") as f:
        paquete = f.read()

    datos_con_meta = descifrar(paquete, password)

    # Recuperar extensión original
    ext = datos_con_meta[:10].rstrip(b"\x20").decode("utf-8")
    datos_doc = datos_con_meta[10:]

    # Si la ruta de salida no tiene extensión, la añadimos automáticamente
    if not os.path.splitext(ruta_salida)[1]:
        ruta_salida = ruta_salida + ext

    with open(ruta_salida, "wb") as f:
        f.write(datos_doc)

    tipo = EXTENSIONES_DOCUMENTO.get(ext, "Archivo genérico")
    tam_mb = len(datos_doc) / (1024 * 1024)
    print("listo.")
    print(f"  Tipo restaurado : {tipo} ({ext})")
    print(f"  Tamaño original : {tam_mb:.2f} MB")
    print(f"  Guardado en     : {ruta_salida}")
```

Toma un documento (".pdf", ".doc", ".docx", ".xls", ".xlsx", ".ppt", ".pptx", ".odt", ".ods", ".odp", ".txt", ".csv", ".xml", ".json", ".zip", ".rar") y las convierte en un documento cifrado y viceversa.

```python
# ────────────────────────────────────────────────────────────────────────────
#  4. CIFRADO DE CONTRASEÑAS Y UBICACIÓN
# ────────────────────────────────────────────────────────────────────────────

def cifrar_credenciales(datos: dict, password: str) -> str:
    datos["_timestamp"] = datetime.utcnow().isoformat() + "Z"
    json_bytes = json.dumps(datos, ensure_ascii=False).encode("utf-8")
    paquete = cifrar(json_bytes, password)
    return base64.b64encode(paquete).decode("ascii")


def descifrar_credenciales(cifrado_b64: str, password: str) -> dict:
    paquete = base64.b64decode(cifrado_b64.encode("ascii"))
    json_bytes = descifrar(paquete, password)
    return json.loads(json_bytes.decode("utf-8"))


def guardar_credenciales_archivo(cifrado_b64: str, ruta: str) -> None:
    with open(ruta, "w", encoding="utf-8") as f:
        json.dump({"blowfish_payload": cifrado_b64}, f, indent=2)
    print(f" Credenciales cifradas guardadas en: {ruta}")


def cargar_credenciales_archivo(ruta: str) -> str:
    with open(ruta, "r", encoding="utf-8") as f:
        data = json.load(f)
    return data["blowfish_payload"]
```

Este bloque lo que hace es guarda datos tipo: usuario, contraseña, sitio web y ubicación GPS

La interfaz del flujo

```python
# ────────────────────────────────────────────────────────────────────────────
#  INTERFAZ DE LÍNEA DE COMANDOS ═══════════════════════════════════════════════════════════════════════
# ────────────────────────────────────────────────────────────────────────────

def banner():
    print("\n BLOWFISH - Algoritmo: Blowfish CBC + PBKDF2-SHA256 \n")
    exts = "  ".join(EXTENSIONES_DOCUMENTO.keys())
    print(f"  Documentos soportados: {exts}\n")


def menu_principal():
    print("  ¿Qué deseas cifrar?")
    print("  [1] Texto")
    print("  [2] Imagen")
    print("  [3] Documento (PDF, Word, Excel, PowerPoint...)")
    print("  [4] Contraseña / Ubicación GPS")
    print("  [0] Salir")
    return input("\n  Elige opción: ").strip()
```

En la interfaz se va a representar el panel donde se mostrará las opciones para poder seleccionar.

Texto

```python
def flujo_texto():
    print("\n── TEXTO ──────────────────────────────────────────────")
    print(" [C] Cifrar  |  [D] Descifrar")
    accion = input("  Acción: ").strip().upper()

    if accion == "C":
        password = pedir_password_nueva("  Contraseña maestra")
        texto = input("  Escribe el texto a cifrar:\n  > ")
        resultado = cifrar_texto(texto, password)
        print(f"\n  Texto cifrado (copia este bloque):\n")
        for i in range(0, len(resultado), 64):
            print("  " + resultado[i:i+64])
        guardar = input("\n  ¿Guardar en archivo? (s/n): ").strip().lower()
        if guardar == "s":
            ruta = input("  Nombre del archivo [cifrado.txt]: ").strip() or "cifrado.txt"
            with open(ruta, "w") as f:
                f.write(resultado)
            print(f"  Guardado en {ruta}")
```

En esta parte se va a mostrar lo que se va a imprimir si seleccionamos la opción 1 y nos pedirá que coloquemos una contraseña maestra, da ahí si escogemos la opción c nos guiara a un apartado donde vamos a escribir lo que se va a encriptar para luego preguntar si lo quiere que se convierta en un archivo

```python
elif accion == "D":
    password = pedir_password_existente("  Contraseña maestra")
    op = input("  ¿Tienes archivo? (s/n): ").strip().lower()
    if op == "s":
        ruta = input("  Ruta del archivo: ").strip()
        try:
            with open(ruta, "r") as f:
                cifrado_b64 = f.read().strip()
        except FileNotFoundError:
            print(" Archivo no encontrado")
            return
    elif op == "n":
        print("  Pega el texto cifrado (Base64), luego presiona Enter dos veces:")
        lineas = []
        while True:
            linea = input()
            if linea == "":
                break
            lineas.append(linea.strip())
        cifrado_b64 = "".join(lineas)
    else:
        print("  Opción no válida")
        return
    try:
        resultado = descifrar_texto(cifrado_b64, password)
        print(f"\n  Texto descifrado:\n  {resultado}")
    except Exception:
        print("\n Error: contraseña incorrecta o datos corruptos.")
else:
    print("  Opción no válida.")
```

Si seleccionamos d nos pedirá la contraseña maestra y nos guiara a una parte donde pregunta si tenemos un archivo o no, si se tiene colocar la ruta donde se aloja el archivo y si no solo pegar el texto encriptado para hacer el proceso de descifrando y poder leer el mensaje.

Imagen

```python
# ═══════════════════════════════════════════
#  FLUJO 2 — Imagen
# ═══════════════════════════════════════════

def flujo_imagen():
    print("\n── IMAGEN ──────────────────────────────────────")
    print("  [C] Cifrar  |  [D] Descifrar")
    accion = input("  Acción: ").strip().upper()

    if accion == "C":
        password = pedir_password_nueva("  Contraseña maestra")
        entrada = input("  Ruta de la imagen original (ej: foto.jpg): ").strip()
        salida  = input("  Ruta del archivo cifrado   (ej: foto.bf):  ").strip()
        if not salida:
            salida = os.path.splitext(entrada)[0] + ".bf"
        try:
            cifrar_imagen(entrada, salida, password)
        except FileNotFoundError as e:
            print(f" {e}")
        except Exception as e:
            print(f" Error al cifrar: {e}")

    elif accion == "D":
        password = pedir_password_existente("  Contraseña maestra")
        entrada = input("  Ruta del archivo cifrado      (ej: foto.bf):     ").strip()
        salida  = input("  Ruta de salida sin extensión  (ej: foto_dec):    ").strip()
        if not salida:
            salida = os.path.splitext(entrada)[0] + "_descifrado"
        try:
            descifrar_imagen(entrada, salida, password)
        except Exception:
            print("\n Error: contraseña incorrecta o archivo corrupto.")
    else:
        print("  Opción no válida.")
```

En esta parte se va a mostrar lo que se va a imprimir si seleccionamos la opción 2 y nos pedirá que coloquemos una contraseña maestra, da ahí si escogemos la opción c nos guiara a un apartado donde vamos a pegar la dirección en donde se encuentra nuestra

imagen y colocar otra ruta para dejar el archivo que esta encriptado .bf. Si seleccionamos d nos pedirá la contraseña maestra y nos guiará a una parte donde tenemos que pegar la ruta donde está el archivo encriptado y otra ruta donde vamos a dejar el la imagen desencriptada y poder ver la imagen.

Documento

```python
# ────────────────────────────────────────────────────────────
#  FLUJO 3 — Documento
# ────────────────────────────────────────────────────────────

def flujo_documento():
    print("\n── DOCUMENTO ──────────────────────────────────────────────")
    exts_lista = ", ".join(EXTENSIONES_DOCUMENTO.keys())
    print(f"  Formatos soportados: {exts_lista}")
    print("  [C] Cifrar  |  [D] Descifrar")
    accion = input("  Acción: ").strip().upper()

    if accion == "C":
        password = pedir_password_nueva("  Contraseña maestra")
        entrada = input("  Ruta del documento original  (ej: informe.pdf): ").strip()

        # Sugerir nombre de salida automáticamente
        nombre_base = os.path.splitext(entrada)[0]
        salida_sugerida = nombre_base + ".bfdoc"
        salida = input(f"  Ruta del archivo cifrado     [{salida_sugerida}]: ").strip()
        if not salida:
            salida = salida_sugerida

        try:
            cifrar_documento(entrada, salida, password)
        except FileNotFoundError as e:
            print(f"\n  ✗ {e}")
        except Exception as e:
            print(f"\n  ✗ Error al cifrar: {e}")
```

En esta parte se va a mostrar lo que se va a imprimir si seleccionamos la opción 3 y nos pedirá que coloquemos una contraseña maestra, da ahí si escogemos la opción c nos guiara a un apartado donde vamos a pegar la dirección en donde se encuentra nuestro documento y colocar otra ruta para dejar el documento encriptado.

```python
    elif accion == "D":
        password = pedir_password_existente("  Contraseña maestra")
        entrada = input("  Ruta del archivo cifrado     (ej: informe.bfdoc): ").strip()

        nombre_base = os.path.splitext(entrada)[0]
        salida_sugerida = nombre_base + "_descifrado"
        salida = input(f"  Ruta de salida (sin extensión) [{salida_sugerida}]: ").strip()
        if not salida:
            salida = salida_sugerida

        try:
            descifrar_documento(entrada, salida, password)
        except FileNotFoundError as e:
            print(f"\n  ✗ {e}")
        except Exception:
            print("\n  ✗ Error: contraseña incorrecta o archivo corrupto.")

    else:
        print("  Opción no válida.")
```

Si seleccionamos d nos pedirá la contraseña maestra y nosotros debemos pegar la dirección donde esta nuestro documento y nos dará el documento ya desencriptado.

Contraseña / Ubicación GPS

```python
# ──────────────────────────────────────────────
#  FLUJO 4 — Contraseña / Ubicación GPS
# ──────────────────────────────────────────────

def flujo_credenciales():
    print("\n── CONTRASEÑA / UBICACIÓN GPS ──────────────────")
    print("  [1] Cifrar contraseña")
    print("  [2] Cifrar ubicación GPS")
    print("  [3] Descifrar archivo de credenciales")
    sub = input("  Opción: ").strip()

    if sub == "1":
        password = pedir_password_nueva("  Contraseña maestra (para el cifrado)")
        datos = {}
        datos["sitio"]      = input("  Sitio web / aplicación: ").strip()
        datos["usuario"]    = input("  Usuario / email: ").strip()
        datos["contraseña"] = input("  Contraseña a guardar: ")
        datos["notas"]      = input("  Notas adicionales (opcional): ").strip()
        cifrado = cifrar_credenciales(datos, password)
        ruta = input("  Nombre del archivo de salida [credencial.json]: ").strip() or "credencial.json"
        guardar_credenciales_archivo(cifrado, ruta)
```

En esta parte se va a mostrar lo que se va a imprimir si seleccionamos la opción 4 y nos pedirá que coloquemos una contraseña maestra, da ahí si escogemos la opción c nos guiara a un apartado nos preguntara que coloquemos un sitio, usuario, contraseña y nota (opcional) esto hace referencia a las contraseñas para acceder a un sitio y en automático nos crea un archivo encriptado.

```python
    elif sub == "2":
        password = pedir_password_nueva("  Contraseña maestra (para el cifrado)")
        datos = {}
        datos["tipo"]         = "ubicacion_gps"
        datos["latitud"]      = input("  Latitud  (ej: 19.4326): ").strip()
        datos["longitud"]     = input("  Longitud (ej: -99.1332): ").strip()
        datos["descripcion"]  = input("  Descripción del lugar: ").strip()
        datos["notas"]        = input("  Notas adicionales (opcional): ").strip()
        cifrado = cifrar_credenciales(datos, password)
        ruta = input("  Nombre del archivo de salida [ubicacion.json]: ").strip() or "ubicacion.json"
        guardar_credenciales_archivo(cifrado, ruta)

    elif sub == "3":
        password = pedir_password_existente("  Contraseña maestra (para el cifrado)")
        ruta = input("  Ruta del archivo cifrado (.json): ").strip()
        try:
            cifrado_b64 = cargar_credenciales_archivo(ruta)
            datos = descifrar_credenciales(cifrado_b64, password)
            print("\n  Datos descifrados:")
            for k, v in datos.items():
                if k != "_timestamp":
                    print(f"    {k:15}: {v}")
            print(f"    {'cifrado el':15}: {datos.get('_timestamp', '?')}")
        except FileNotFoundError:
            print(f"  Archivo no encontrado: {ruta}")
        except Exception:
            print("\n  Error: contraseña incorrecta o archivo corrupto.")
    else:
        print("  Opción no válida.")
```

Si se selecciona la opción 2, podemos hacer lo mismo de encriptar como la anterior pero aquí nos pide latitud, longitud, descripción y nota (opción) y crea el archivo encriptado y por último si seleccionamos la opción 3 esta parte lo que hace es desencriptar las 2 cosas que se mencionaron, solo pedimos la contraseña maestra y la ruta del archivo encriptado para que te dos datos.

```python
def main():
    banner()
    while True:
        opcion = menu_principal()
        if opcion == "1":
            flujo_texto()              # ← Se implemento texto
        elif opcion == "2":
            flujo_imagen()             # ← Se implemento las imagenes
        elif opcion == "3":
            flujo_documento()          # ← Se implemento los documentos
        elif opcion == "4":
            flujo_credenciales()       # ← Se implemento los documentoslas credenciales
            print("\n  Hasta luego. Mantén tus datos seguros.\n")
            break
        else:
            print("  Opción no válida, intenta de nuevo.")
        print()


if __name__ == "__main__":
    main()
```

En este último bloque solamente va a señalar la lógica del programa que se va a seleccionar.

Cambios del código original a la versión modificada

En esta sección se presentan las mejoras realizadas al código original, enfocadas en hacerlo más organizado, funcional y completo. Se pasó de un programa que solo cifraba texto a una herramienta capaz de manejar distintos tipos de datos, como imágenes, documentos y credenciales, incorporando nuevas funciones y librerías que facilitan su uso y mantenimiento.

Lo que se quedó IGUAL

- El algoritmo Blowfish en modo CBC
- El uso de PBKDF2 con SHA-256 para derivar la clave
- El uso de:
  - salt aleatorio (16 bytes)
  - IV aleatorio (8 bytes)
- La estructura del paquete cifrado:
  - longitud del salt, salt, IV (Vector de Inicialización para descifrando) y datos cifrados
- Las funciones base:
  - derivar_clave, cifrar y descifrar
- El proceso lógico:
  - cifrar →empaquetar →codificar (Base64 en texto)
  - descifrar →desempaquetar →decodificar

Lo que CAMBIÓ

- De solo texto → ahora:
  - texto
  - imágenes
  - documentos
  - credenciales
- Se agregó validación de contraseñas seguras
- Se añadió manejo de archivos (leer, escribir, rutas)

- Se incorporó soporte para múltiples formatos de documentos
- Se agregó almacenamiento de metadatos (extensión del archivo)
- Se implementó guardado en archivos (.txt, .json, .bf, etc)
- Se creó un sistema para cifrar credenciales y ubicación GPS
- Se mejoró la interfaz de la pantalla.
- Se dividió el programa en módulos/funciones por tipo de dato
- Se agregaron nuevas librerías.

Observaciones

- Al cifrar el mismo texto con la misma contraseña en distintas ejecuciones, el resultado Base64 siempre es diferente, gracias al uso de salt e IV aleatorios.
- Si se ingresa una contraseña incorrecta al descifrar, el programa muestra el mensaje de error: 'Error: contraseña incorrecta o datos corruptos.' sin revelar información adicional.
- El proceso de derivación de clave (PBKDF2 con 100,000 iteraciones) introduce un retardo perceptible pero intencional, que dificulta los ataques de fuerza bruta.
- El padding PKCS7 incrementa ligeramente el tamaño del texto cifrado para que sea múltiplo de 8 bytes (tamaño de bloque de Blowfish).
- La instalación inicial de pycryptodome requirió usar 'py -m pip' en lugar de 'pip' directamente, ya que Python no estaba agregado al PATH del sistema.

Conclusiones

La implementación del cifrador Blowfish en Python demostró ser una experiencia práctica valiosa para comprender los principios del cifrado simétrico moderno. A través del desarrollo del programa, se pudieron observar en funcionamiento los conceptos teóricos de bloques, modos de operación, derivación de claves y codificación de datos.

El uso del modo CBC junto con IV aleatorio y PBKDF2-SHA256 eleva significativamente el nivel de seguridad respecto a un cifrado Blowfish básico, protegiendo contra ataques de repetición y de diccionario. Esta combinación de técnicas es representativa de cómo se implementa el cifrado en aplicaciones reales.

Aunque Blowfish es considerado seguro para la mayoría de los casos de uso, su tamaño de bloque de 64 bits lo hace susceptible al ataque SWEET32 en contextos de cifrado de grandes volúmenes de datos, por lo que para nuevos sistemas se recomienda el uso de AES o Twofish (sucesor de Blowfish).

Bibliografía

Schneier, B. (1994). Description of a New Variable-Length Key, 64-Bit Block Cipher (Blowfish). Fast Software Encryption, Cambridge Security Workshop Proceedings. Springer.

Legrandin, L. (2023). PyCryptodome Documentation (v3.23.0). Recuperado de https://pycryptodome.readthedocs.io/

NIST. (2001). FIPS PUB 197 - Advanced Encryption Standard (AES). National Institute of Standards and Technology.

Anexos

Blowfish:

Cifrador de bloques simétrico de 64 bits con clave variable, diseñado por Bruce Schneier en 1993.

Red de Feistel:

Estructura de cifrado que divide el bloque en dos mitades y aplica una función en rondas, intercambiando las mitades en cada iteración.

CBC (Cipher Block Chaining):

Modo de operación que encadena bloques mediante XOR con el bloque cifrado anterior, garantizando que bloques iguales produzcan cifrados distintos.

IV (Vector de Inicialización):

Valor aleatorio usado en el primer bloque CBC para garantizar que el mismo texto nunca produzca el mismo cifrado.

S-box (Substitution box):

Tabla de sustitución usada en la función F de Blowfish para introducir no linealidad en el cifrado.

Padding PKCS7:

Técnica de relleno que completa el último bloque de datos con bytes cuyo valor indica cuántos bytes se agregaron.

XOR:

Operación lógica bit a bit usada para combinar bloques en CBC y en la red de Feistel de Blowfish.